

File System Part I ([previous version of notes](#))

Preliminaries

Why discuss files first?

- Because we know something about file systems, it is more comfortable to start with it.
- [The file system is the foundation for other services in the UNIX.](#) E.g. pipes and internet communications.
- It is important to understand how the UNIX file system works to be able to take advantage of some of its other tools.
- Note: the text talks about a lot more features. I will mention some in class, but we don't time to talk about everything. This will be true for most topics.

In class, we will review the disk hardware fundamentals. Format of disk, seek, latency, transfer rate, caching. You can look at the notes for the Software Design class for more detail.

Directory Structure in UNIX

The following are some basic directories. They are not consistent across all UNIX systems. I will also not talk much about directories that we will not use. If time, we will display the contents of some of the directories.

- `/` : the root directory. This is the starting point for entire file system.
- `/bin` : were most of the commands and system programs may be found. Note: commands are small programs.
-> Examples `mkdir`, `ls`, `ed`. `/sbin` are system programs used by administrators. No available to us.
- `/etc` : Just what the name implies. A place to put a whole bunch of stuff.
-> Examples: `passwd` and `hosts` file.
- `/usr` - user installed programs. It also has subdirectories for libraries, and applications. For example `NX` is there. This is NoMachine.
- `/home` - Information associated with users.
-> For example: there are student and faculty sub-directories. (not this year) Let give it a look. [Note the interesting directory structure for students.](#)
`/usr` (Not there anymore)
- `/usr/include` : where many of the standard include files may be found. Like `iostream`
- `/tmp` : temporary `/tmp` directory. What is stored there should go away on boot up.
- `/dev` : description of devices. For drivers.

Basic system calls that we expect most file systems to provide.

We will discuss in detail later.

- `create` (For UNIX this is spelled `creat` - why?)
- `open` On many file systems, including Linux, the `open` also serves the role of `create` (Linux has a `create` call but few use it. It is in mostly legacy code.)
- `read`
- `write`
- `close`
- `seek`
- `delete` - remove the file - be careful with this. You can delete a lot more than you want - be careful. (For UNIX this is named "unlink")
- `stat` - get data about a file. What kind of data do you think we can get?

I will give general discussion of each system call in class.

Notes on the UNIX file system:

1. [UNIX see a file as a sequence of bytes.](#) The UNIX operating system does NOT see records or lines or characters. They keep things simple. File systems in programming languages worry about these things.
2. [Files accesses are not buffered](#) within your program for these file system calls. Most files systems implemented for programming languages do buffer input and output. Most do allow you to flush the buffers. (This is still not always enough control.) Let's discuss the pros and cons of this in class.
3. [Writing to and reading from files is atomic.](#) What does this mean? This can have big benefits for some applications.
4. [UNIX makes use of disk caching.](#) Careful on this in that it is good news for performance, but bad news in a few cases. We will discuss how you can get false results when testing. But there are tools to deal with these issues.
5. There are also C and C++ files systems. These are of course implemented on UNIX using the UNIX file system. [Why use the UNIX file system rather than the file system provided in a programming language? Why use the file system provided by the programming language?](#)
6. [File names may be made up of any characters.](#) For example `"**"` and `" "` may be part or all of file names. You can even have the ASCII code to ring the "bell". So, you could create a file name that will make a noise any time someone does a directory listing. (It is fun to be annoying) Hard to remove these files using the `rm` command. The names are case sensitive. This is different than windows.

7. Microsoft has implemented the basic UNIX file system calls. (Actual they are POSIX calls.) So, if you ever need unbuffered I/O in windows, these will do it for you. I should of by now, explained why we might need unbuffered I/O in class.

We access these file services through UNIX system calls. These look like C function calls, because they are. Is this a problem?? It does require a way to identify an open file since we don't have an object of a class to serve this purpose. They use an integer which is referred to as a **file descriptor**.

The following is an example of a file system call.

```
nb = read( fd, buff, NumBytes );
```

System calls "talk" to the operating system. Why is this important to know? Because a system call requires a context switch to and from the operating system, so there is additional overhead associated with a system call like the read.

Recall how errors are returned in UNIX:

- Rule: **UNIX usually returns -1 if there is an error.** For functions that return addresses, NULL is usually returned if there is an error. **Why don't they throw exceptions for errors? (Yes, I am repeating.)**
- The external int errno contains a code that describes the error. (This is a global variable) **You MUST always check for errors on each system call we use. I will not accept your lab if you don't do this and you will lose points on tests.**
- I will make sure that everyone understands external variables. The errno variable is an external variable. (Actually it now just appears to be.)

Let's look at errno.h to see what is there. Note there was an **extern int errno;** statement. Now there is a define for errno that replaces it with a function: **#define errno (*__errno_location ())** Note that errno can be a function. Why? Note: the errno.h file contains other include files. These contain the defines for the error codes.

Error Reporting

Error reporting is essential. When a function in the API fails, you want to find this out and why. We will use the **perror** function to report errors. Let's look this function up in the man pages. There is the "man" command to look things up. But, it is a lot easier to do this on the site:

<http://linux.die.net>

Note: There are other sites for this data.

The perror function reports the error number, a description of the error and your message. **I expect to see you using perror on your quizzes and programming assignments.**

Previous to this perror function you had to write your own to do it. **Those days are long gone.** See below for an example of the type of function you would have to write. We will talk about it for the sake of its ideas. It would be strange if you used it.

- errno contains a number that stands for the error.
- It is usually better to have a text error message than an error number. "sys_errlist" is an array of pointers to ASCII strings that describe the errors. "sys_nerr" is the size of this array. These are global variables. **Why would we want to do this?** Note: this is a good tool if you are writing error messages to a log file.

The following is a function could be used to display errors. Note that the man pages list error returns from system calls. We are discussing this function because it opens up a discussion of a number of ideas. I will use perror in the class examples and expect you to use it in your programming assignments and quizzes.

```
#include <iostream>
using namespace std;
```

```
void syserr( const char *msg ) /* print sys error message and terminate. */
{
    extern int errno, sys_nerr;
    extern char *sys_errlist[];

    cerr << "ERROR: " << msg << "(" << errno;

    if( errno > 0 && errno < sys_nerr ) {

        cerr << " - " << sys_errlist[errno] << ")" << endl;
    } else {

        cerr << ")" << endl;
    }
    exit(1);
}
```

Why cerr instead of cout?

We will not use this function. It make no sense to write our own function when there is a standard function that does almost exactly the same job.

See [ex1.cpp](#) for an example of how perror works. I will use the debugger in to demonstrate this function. Also, discuss the basics of make. See [Makefile](#) file for the make file for this project. **Note: there will be an error when we compile due to the make file so we can see how to deal with it.** We will also create this program in Eclipse. Note to me: my initial examples are in the workspace called "workspace".

Note: it is not a good move to name your program test.

Example of use

The following is an example of more sophisticated file use to get a perspective on why these calls are very useful.

- Discuss the problem of accessing minute data from FasTick. They kept market data in quotes and trades. So, to get minute data, you had to call functions read through all of this data.
- Picture simulations that require 500 days of 1 minute bar data with a thousands stocks.
- By using the FasTick product directly, it would take about 10 hours to access the underlying data and compute the minute bars for this usage. So each simulation involving this data would therefore take more than 10 hours to run.
- We need an alternative approach. What approach do you think I came up with? Finding alternate solutions is what makes you valuable to your company.

The file access system calls Quick Review starting here: 9/10/2025

- Let's go through the basic system calls to do file access. Namely: open, read, write, lseek, close.
- We are diving right in rather than given background material first.
- Note: the textbook supplies a lot of detail that we do not have time to talk about. Too many details can make learning more difficult instead of easier.
- We will come back to things like how the file system is set up.
- Note: Windows have a similar set of unbuffered calls that you might want to use if you need this functionality. (Yes, I am repeating this)

The Linux file system is important because large parts of the operating system use it. For example, process to process communications and network programming go through the file system. So, the file system is used for a lot more than just accessing data from files.

open call

The first systems call presented will be the **open**.

- You can find all the information I am discussing (and more) using the man command or by reading the text or looking this up on the web.
- Note: the following link is very useful for man pages: <http://linux.die.net/man/> (Yes, I am repeating myself.)
- The prototypes can vary a bit between Linux/UNIX implementations. Mostly it is just a matter of the names for the arguments and argument types. The actual way that they work does not tend to vary.
- I will show the help pages using the link I gave. This is easier than using the man command.
- The open call prepares your program to access a file. What does this mean??

Note: there is an ancient call called **creat** to create new files. We will not talk about it, but you might run into it. It is used to create a file. The creat function has a similar format to the open function and has been made obsolete by the open call options. Someone who uses it has probably been around for a long time or the code they are working on has been.

```
#include < sys/types.h >
#include < sys/stat.h >
#include <fcntl.h >
#include < limits.h > /* definition of OPEN_MAX */
```

```
int open( const char * path, int flags, mode_t mode)
```

path : the name of the file optionally including a full directory path. If no path is given, the file is assume to be in the working directory from which the program was executed. Notice that this is a C style string. We can use the `c_str()` method from the string class if we want to use C++ strings.

flags : defines how the file is to be opened. This argument is formed by OR'ing the following values (I will not list all of them) OR'ing values is typical for Linux options.

- O_RDONLY : Open for reading only.
- O_WRONLY : Open for writing only.
- O_RDWR : Open for reading and writing.
- O_APPEND : Append to the end of the file on each write.
- O_CREAT : Create file if it does not exist.
- O_TRUNC : Truncate size to 0. Allows a cleanup before writing to a file.
- O_EXCL : Error if trying to create a file and file exists. Why would you want this option?
- O_DSYNC : Do file writes synchronously. That is each write must fully complete before any other operation occurs. (This means that the data is on the disk drive) This guy is super safe, but a lot slower. Note: the writes are atomic.
- O_CLOEXEC : means that if you schedule another process, it will not inherit the open file descriptor. (We will see a need for this later.)
- O_DIRECT - eliminates caching. Why would you want to do this? This is writing directly to the file.

Make sure everyone recalls how to do bit wise OR's using the "|" operator. Note: some flags are system dependent. It is always good to choose the least level of system dependent flags. Why?

mode : this parameter is optional. (How does this happen? Optional parameters used to be trickier to implement before there were default arguments. C has tools for dealing with a variable number of arguments. This differs from default arguments of C++.) The mode argument is only used if the file is created by this call. (So, if the call has O_CREAT in the flags and the file did not previously exist.) The mode is used to describe **permissions** associated with the file In the old days people used a 3 digit octal number to describe the permission. The digits correspond to owner, group, and other. The bits within a digit indicate read, write, and execute permissions. How do you specify an octal number in C++? To give read and write permission to the owner only, mode = 0600.

Lets look at /usr/include/sys/stat.h for newer way of specifying permissions. I will stick with the old way. Usually the new way is better, but at least for me, the old way is quicker to write and almost everyone understands it. The following is a section from the stat.h file. You "or" together the permissions you want.

```
#define S_IRWXU 00700 /* read, write, execute: owner */
#define S_IRUSR 00400 /* read permission: owner */
#define S_IWUSR 00200 /* write permission: owner */
#define S_IXUSR 00100 /* execute permission: owner */
#define S_IRWXG 00070 /* read, write, execute: group */
#define S_IRGRP 00040 /* read permission: group */
#define S_IWGRP 00020 /* write permission: group */
#define S_IXGRP 00010 /* execute permission: group */
#define S_IRWXO 00007 /* read, write, execute: other */
#define S_IROTH 00004 /* read permission: other */
#define S_IWOTH 00002 /* write permission: other */
#define S_IXOTH 00001 /* execute permission: other */
```

Make sure that if you specify the O_CREAT flag that you put the mode argument in the open call. Otherwise, life might get interesting. :-) [I will explain why.](#) (It is due to another option called the sticky bit which restricts who can delete a file from a directory.)

Returns -1 if there is an error and a nonnegative integer (file descriptor) if the function succeeds. All subsequent system calls that reference this file will use this file descriptor.

Notes:

1. [File descriptors 0, 1, and 2 are pre-assigned.](#) These correspond to the keyboards, screen, and screen (for error messages) respectively. If you close file descriptor 1, you will cause interesting effects. For example, the cout object will no longer work. Why? [Let demo this.](#)
2. The previous statement has interesting implications. It means that access to a screen or from a keyboard goes through file descriptors in the same manner as a file. This is also true of other means of communications and other devices. This give use the ability to redirect output from the screen to a file.
3. A File descriptor represent a position in a table that is local to the process.
4. The previous note implies that file descriptors are not global across processes. That means file descriptor 4 in one process has no relationship to file descriptor 4 in another.
5. [File descriptors are assigned lowest available number.](#) So, on your first open, your should see a File descriptor having the value of 3. (Not always if you run in the debugger - why?)
6. There is a limit on how many files may be opened by a single process. Many Linux systems have 64K (was 1024 previously) as the limit. How would you find this out? Look it up online. The command is: [ulimit -n](#) This limit can be changed for the system. Ours, as of this writing, has a limit of 16384. This is a good thing for safety. It used to be pretty small - like 64.
7. If you use the equivalent calls in Microsoft OSs, be aware that the file descriptors are not assigned sequentially and are not small numbers.
8. To repeat myself, there is a separate function to create files called [creat](#). This function is less powerful than open and obsolete, so it should not be used. It continues exist for the sake of maintaining old code. It has many of the same arguments as the open call. If you in a group supporting legacy code, why would you not replace creat calls with open calls?

read call

The read function reads data from an open file start with the current file offset. It has the following format:

```
#include <unistd.h>
ssize_t read(int fd, void *buff, size_t nbytes);
```

fd : the file descriptor for an open file.

nbytes : the number of bytes to be read.

buff : address of a buffer into which the data will be read.

What is size_t? What is ssize_t. [What the heck are they thinking! I will explain.](#)

Returns: If the returned value is 0, then end-of-file has been reached. If the read is successful, the number of bytes actually read is returned. Otherwise, a -1 is returned and the global variable errno is set to indicate the error.

Note about where in the file we are reading from and what the "current file" offset or "file pointer" refers to.

write call

The write system call is used to write data to a file. It has the following format:

```
#include <unistd.h>
ssize_t write(int fd, const void *buff, size_t nbytes);
```

fd : the file descriptor for an open file.

nbytes : the number of bytes to be written.

buff : buffer for the data to be written.

This function returns the number of bytes written to the file. It returns -1 if there was an error. **Question: why the use of const in the prototype for buff?**

Note: this call is atomic as is the read. Recall what this means. A note: if two separate processes have file open, only one can read or write at one time.

close call

The **close** function allows the operating system to recover the resources that were allocated when the file was opened. All the files that your program has opened are automatically closed when your program terminates. **So, why close files? Not closing files that are no longer accessed can cause big problems if there are lots of files and the process stays open.**

The close function has the following format:

```
#include <unistd.h>
int close(int fd)
```

fd : the file descriptor of an open file.

This function returns 0 if it is successful and -1 if it fails. Why would it fail?

lseek call

- Sometime you might not want to read sequentially file. Why? (I will give a CSV file example where we want to read the last line. But, this is more with using C++ or C read calls. There are other examples.) Another case if if you have binary files with an index at the beginning.
- The lseek function allows you to position the current file offset so that you can read from or write to anywhere in the file. You might wonder why this wasn't called seek. Any ideas?
- If you try to read when pointing beyond the end of the file what do you think happens? You get a zero length read.
- If you attempt to write beyond the end of a file what do you think happens? We will demonstrate later with example ex3. It is pretty strange.

The format of the lseek is as follows:

```
#include < sys/types.h >
#include < unistd.h >
off_t lseek( int fd, off_t offset, int whence)
```

fd: a file descriptor from an open file.

offset: a file offset. This is a signed integer value.

whence: indicates what the offset is from.

- If whence is SEEK_SET, the pointer is set to offset bytes.
- If whence is SEEK_CUR the pointer is set to its current location plus offset.
- If whence is SEEK_END, the pointer is set to the size of the file plus the offset.

Note: the offset may be positive or negative.

Returns the new file offset if successful and -1 if fails.

Note: lseek can be used to find the current location. How? It can be used to find the size of the file. How?

Examples:

Show [ex1.cpp](#) again as an example using perror. Also run in debugger.

[ex2.cpp](#) and [copy.cpp](#) - is an example of a function to copy the contents of one file into another. Note the way I can use the debugger to find why a program core dumps. (Note F8 means resume execution.) We will also create this executable without using Eclipse. Note: "tail -f <file name>" will be useful in testing in that it will display data as it is added to the file.

[ex3.cpp](#) Shows the use of lseek and what happens when you write beyond the end of a file. Note about how this can be used to mess up people who do backups. :-)

DO NOT DO THIS! Depending on how the system calculates the space you have used, you could run out of your allocated space

Lab1

Write a program called rtail which will take a file name as its command line argument and display the last 10 lines in the file. Don't make any assumptions on the size of the file you are displaying. You are to use the UNIX system calls. Make this program as **efficient** as possible. We will discuss more in class. **To get it right, this lab takes more coding time that you would expect, so start it right away.**

Note: In class we will determine how lines of text in a file are delimited. We will use the od command. This allows us to dump in octal and other formats. You could use: od -ax <file name> A better command is: hexdump -cx <file name>. This does the same thing.

If there is time, write an example in class that will display the contents of a text file in reverse order.

I/O Efficiency

Recall that disk data is cached. So, this does not mean we have to read from the disk every time we want data. The text has a nice table (3.6) that I would like to discuss. It give measures of speed in accessing data from a large file (516,581,760 bytes) through Linux. It uses a variety of buffer sizes. Please read the text about this table and how they did the experiment. Note, they are doing fresh reads from the disk. Why?

Buff Size	User CPU Time	System CPU Time	Clock Time	#Loops
1	20.03	117.50	138.73	516,581,760
2	9.69	58.76	68.60	258,290,880
128	0.34	1.84	8.69	4,035,795
1024	0.02	0.78	8.58	504,475

4096	0.03	0.58	8.62	126,119
16,384	0.01	0.56	8.69	31,530
524,288	0.01	0.58	5.58	986

**Note:**

Reads and writes are atomic. That is they will not be interrupted once started. That is a write to a file will be allowed to complete before another read or write operation on the same file can occur. (How is this accomplished - by the operating system?) This is important for multithreaded and multi-process applications.

The pread and pwrite function calls

These calls combine seek and a read or write in a single atomic operation. Why??? Let's look at the description of these system calls. See: <http://linux.die.net> You can specify the number of bytes to read or write and the offset into the file.

The readv and writev function calls

These calls allow us to read and write vectors of data in a single atomic operation. Why?? Let's look at the description of these system calls.